NUST Institute of Civil Engineering (NICE-SCEE)
National University of Sciences & Technology (NUST)

CS114: Fundamentals of Programming

Class: BECE2018

Lab 05: First Program, Variables, Arithmetic Operators, Stream Operators and Escape Sequences

Instructor: Qurrat-ul-ain Babar

Lab 05: First Program, Variables, Arithmetic Operators, Stream Operators and Escape Sequences

The aim of this lab is to familiarize student with the programming environment and cover some other basics of C++ programming.

Our aims today are:

1. To learn
   - C++ Programming Environment
   - Variable Types
2. To learn output in C++
   - cout
3. To learn C++'s Comments
4. To learn about Arithmetic Operators
5. Stream Operators
6. Use of Escape Sequences

Deliverables

1. Execute all **Examples** given in this document and share screen shots of the outputs of each example.
2. Execute all **Tasks** given in this document and share readable code and take screen shots of the outputs of each task.

Upload single word document on LMS. Name and Save your file as Firstname_Lastname_Regno. No late submissions will be accepted.

Deadline: Check deadline of the submission link on LMS.

Task list is given at the end of the lab manual.

Before we move on to the actual lab, let's look at a simple program that displays the following statement as output:

Welcome to C++

Example 1:

Run below given program and display output

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Welcome to C++";

    return 0;
}
```

Variables, Types and Operators

A variable is a place to store a piece of information. Just as you might store a friend's phone number in your own memory, you can store this information in a computer's memory. Variables are your way of accessing your computer's memory.
C++ imposes fairly strict rules on how you can name your variables:

- variable names must begin with a letter
- variable names are "case-sensitive" (i.e., the variable "myNumber" is different from the variable "MYNUMBER" which is different from the variable "mYnUmBeR")
- variable names can't have spaces
- variable names can't have special characters (typographic symbols)

What can you name your variables? In general, variable names can be composed of **letters**, **numbers**, and **underscores** (_). However, C++ reserves certain keywords which have special meaning to the language, and you are not allowed to use any of these keywords as variables names. Some examples of C++ keywords are **int**, **for**, **else**, and **class**. You can, however, use keywords in the middle of a variable name, such as "**foreign**" or "**classical**".

Variable Types

A variable type is a description of the kind of information a variable will store. Following are some of the Types that a variable can have:

Data Type	Keyword Used	Example	Declarations
Integer (Whole Numbers)	Int	21	int a; int b=21;
Floating Point (Decimal Numbers)	Float	1.56	float a; float b=1.56;
Character (Characters)	Char	A	char a; char a='A';

Declaring Variables

Declaring a variable in C++ is simple. Let's say you want to declare a variable of type **int** called **myAge**. That is to say, the variable myAge will store an integer. In C/C++, this is written: **int myAge;**

All this does is tell the computer that you plan to use an integer, and that the integer's name is myAge. In some languages, variables are initialized to 0 - that is, a variable's initial value

will be 0. This is not true of C++! Sometimes your variables will be initialized to 0, but sometimes they will be initialized with garbage. As you might anticipate, this can cause some nasty bugs. Hence, it is always a good idea to initialize your variables with some value. If you don't know what a variable's initial value should be, initialize it to 0. Initializing a variable is easy.
Let's write a program that stores your age in a variable and outputs "My age is 21". The first line of the main function initializes myAge by assigning it a value immediately.

Example 2:

Run below given program and display output.

```
#include <iostream>
using namespace std;

int main()
{
    int myAge = 21;
    cout << "My age is"
         << myAge << " years";

    return 0;
}
```

To print out the value of some variable, you need to embed a format specifier in your text string and pass extra arguments to the cout function. An example:
cout<< "My Age is " <<myAge<< "Years";
This statement prints out the value of the variable myAge. Note that the value of myAge is passed to the cout function.

That's all there is to it! By the way, the equals sign ("=") is called an operator which will be discussed in more detail later. Also, note the way we used cout to output the value of a variable.

Output in C++ (cout)

Gives you the power to print output onto the screen, and is relatively simple to use. On most program environments, the standard output by default is the screen, and the C++ stream object defined to access it is ***cout***.

For formatted output operations, ***cout*** is used together with the insertion operator, which is written as "**<<**" (i.e., two "less than" signs).

```
cout <<"Output sentence"; // prints Output sentence on screen
cout << 120;              // prints number 120 on screen
cout << x;                // prints the value of x on screen
```

The << operator inserts the data that follows it into the stream that precedes it. In the examples above, it inserted the literal string Output sentence, the number 120, and the value of variable x into the standard output stream cout. Notice that the sentence in the first

statement is enclosed in double quotes (") because it is a string literal, while in the last one, x is not. The double quoting is what makes the difference; when the text is enclosed between them, the text is printed literally; when they are not, the text is interpreted as the identifier of a variable, and its value is printed instead. For example, these two sentences have very different results:

```
cout <<"Hello";  // prints Hello
cout << Hello;    // prints the content of variable Hello
```

Multiple insertion operations (<<) may be chained in a single statement:

```
cout <<"This "<<" is a "<<"single C++ statement";
```

This last statement would print the text **This is a single C++ statement**. Chaining insertions is especially useful to mix literals and variables in a single statement:

```
cout <<"I am "<< age <<" years old and my zipcode is "<<zipcode;
```

Assuming the *age* variable contains the value 24 and the *zipcode* variable contains 90064, the output of the previous statement would be:

I am 24 years old and my zipcode is 90064

What cout does not do automatically is add line breaks at the end, unless instructed to do so. For example, take the following two statements inserting into cout:
cout << "This is a sentence.";
cout << "This is another sentence.";

The output would be in a single line, without any line breaks in between. Something like:
This is a sentence.This is another sentence.

To insert a line break, a new-line keyword shall be inserted at the exact position the line should be broken. In C++, the **endl** manipulator can be used to break lines. For example:

```
1 cout<<"First sentence."<<endl;
2 cout<<"Second sentence."<<endl;
```

This would print:

First sentence.
Second sentence.

Example 3:

Display output of the following code:

```
int main()
{
int a = 5;
intc = 't';
cout<< "Value of a is " << a<<endl;
cout<< "and value of c is " << c <<endl;
/*
You can also print the above in only one statement
*/
cout<< "Value of a is " << a << "and value of c is " << c <<endl;

return 0;
}
```

Arithmetic Operators

Arithmetic operators are commonly used in a variety of programming languages. In C, there are five of them, and they all take two **OPERANDS**. Recall that an operand is an expression that is required for an operator to work. For example, for 8 + 4, 8 and 4 are considered as the operands.

Operator Name	Symbol
Multiplication	*
Division	/
Modulus	%
Addition	+
Subtraction	-

Example 4:

Run the below given program and display the output

```
#include <iostream>
#include<conio>

int main()
{
    int a, b, c, sum;
    a=1;
    b=2;
    c=3;

    sum=a+b+(c);

    cout <<"Sum = " <<sum;
    cout<<"C     = "<<c;

    sum=a+b+(c--)

    cout <<"Sum = " <<sum;
    cout <<"C     = "<<c;

    return 0;
}
```

What's With the %?

Multiplication, addition and subtraction are the simplest to use. Division is also easy, but watch out for the truncation of an int divided by an int! Now, the one that confuses novices is the modulus operator, sometimes known as the remainder operator.

To keep things simple, a%b returns the **REMAINDER** that occurs after performing a/b. For this operator, a and b MUST be integers!
For example, 6%3 returns 0 because 3 goes into 6 EXACTLY. Similarly, 4%4, 8%2 and 16%8 all return 0.

So why does 3%4 return 3? Picture it this way: you have 3 holes to fill, but you can only fill 4 holes at a time. You can't fill a group of 4 holes, therefore the 3 holes you had are still empty. Similar story for 7%4 because you can fill in one group of 4 but still have 3 holes remaining.

C++ Comments

A comment is text that the compiler ignores but that is useful for programmers. Comments are normally used to annotate code for future reference. The compiler treats them as white space. You can use comments in testing to make certain lines of code inactive.

A C++ comment is written in one of the following ways:

- The /* (slash, asterisk) characters, followed by any sequence of characters (including new lines), followed by the */ characters. This syntax is the same as ANSI C.
- The // (two slashes) characters, followed by any sequence of characters. A new line not immediately preceded by a backslash terminates this form of comment. Therefore, it is commonly called a single-line comment.

The comment characters (/*, */, and //) have no special meaning within a character constant, string literal, or comment.

Example 5:

Run below given program and display the output

```
int main()
{
//declaring integer and character
vairables
                int a; char ch;
/*Initializing the
 variables
*/
                a=10; ch='b';
/*Printing the
 variables
*/
cout<<"The value of ch is "
<<ch<<endl;
                cout<<"The value
of a is "<<a<<endl;

                return 0;
}
```

In this example you saw various ways of declaring variables of various data types and how to write comments in C/C++ programs.

C++ I/O occurs in streams, which are sequences of bytes. If bytes flow from a device like a keyboard, a disk drive, or a network connection etc. to main memory, this is called ***input operation*** and if bytes flow from main memory to a device like a display screen, a printer, a disk drive, or a network connection, etc., this is called ***output operation***.

The Standard Output Stream (cout)

The predefined object cout is an instance of **ostream** class. The cout object is said to be "connected to" the standard output device, which usually is the display screen. The cout is used in conjunction with the **stream insertion operator**, which is written as **<<** which are two less than signs as shown in the following example.

Example 6

```
#include <iostream>
using namespace std;

int main()
{
   string str = "Hello C++";

   cout << "Value of str is : " << str << endl;

}
```

When the above code is compiled and executed, it produces the following result –

Value of str is : Hello C++

The C++ compiler also determines the data type of variable to be output and selects the appropriate stream insertion operator to display the value. The << operator is overloaded to output data items of built-in types integer, float, double, strings and pointer values.

The insertion operator << may be used more than once in a single statement as shown above and endl is used to add a new-line at the end of the line.

The Standard Input Stream (cin)

The predefined object cin is an instance of **istream** class. The cin object is said to be attached to the standard input device, which usually is the keyboard. The cin is used in conjunction with the **stream extraction operator**, which is written as **>>** which are two greater than signs as shown in the following example.

Example 7

```cpp
#include <iostream>
using namespace std;

int main()
{
   string name;

   cout << "Please enter your name: ";
   cin >> name;
   cout << "Your name is: " << name << endl;

}
```

When the above code is compiled and executed, it will prompt you to enter a name. You enter a value and then hit enter to see the following result.

Note: Here it will only accept the first name if spaces are given within the name. Explore why?

```
Please enter your name: cplusplus
Your name is: cplusplus
```

The C++ compiler also determines the data type of the entered value and selects the appropriate stream extraction operator to extract the value and store it in the given variables.

The stream extraction operator >> may be used more than once in a single statement. To request more than one datum you can use the following −

```
cin >> name >> age;
```

This will be equivalent to the following two statements −

```
cin >> name;
cin >> age;
```

Example 8

Run the program given below and see what will be the output:

```
#include <iostream>
using namespace std;

int main()
{
   int a, b;

   cout << "Enter value of a \n";
   cin >> a;

   cout << "Enter value of b \n";
   cin >> b;

   cout << "The value of a is: " << a << endl;
   cout << "The value of b is: " << b << endl;

   // Taking multiple input separated by space
   cout << "Enter new values of a and b separated by a space \n";
   cin >> a >> b;

   cout << "New values of a and b are: " << a << " " << b << endl;

   return 0;
}
```

Escape Sequences

Escape sequences are special characters used in control string to modify the format of an output. These specific characters are translated into another character or a sequence of characters that may be difficult to represent directly. For example, you want to put a line break in the output of a C++ statement then you will use "\n" character which is an escape sequence itself.

An escape sequence consists of two or more characters. For all sequences, the first character will be "\" i.e. backslash. The other characters determine the interpretation of escape sequence. For example, "n" of "\n" tells the cursor to move on the next line.

Escape sequences	Description
\'	This escape sequence in C++ is used for Single Quote.
\"	This escape sequence in C++ is used for Double Quotes.
\?	This escape sequence in C++ is used for Question Mark.
\n	This escape sequence in C++ is used for New Line.
\t	This escape sequence in C++ is used for Horizontal Tab.
\v	This escape sequence in C++ is used for Vertical Tab.
\a	This escape sequence in C++ is used for Alarm Beep.
\b	This escape sequence in C++ is used for Backspace.
\f	This escape sequence in C++ is used for New Page.
\r	This escape sequence in C++ is used for Carriage Return.

Example 9:

Run the below given program and observer the behavior of each escape character.

```
#include<iostream>

using namespace std;

int main()
{
    cout << "This\nis\na\ntest\n\nShe said, \"How are you?\"\n\n";
    //  cout<< "Hey, \"How are you\?\"" ;

    cout << "COMPUTER\tSCIENCE";        // shows the result of \t
    cout << "\n\n";
    cout << "COMPUTER\aSCIENCE";        // shows the result of \a
    cout << "\n\n";
    cout << "COMPUTER\bSCIENCE";        // shows the result of \b
    cout << "\n\n";
    cout << "\'COMPUTER SCIENCE\'";     // shows the result of \'
    cout << "\n\n";
    cout << "COMPUTER\rSCIENCE";        // shows the result of \r
    cout << endl;

    return 0;
}
```

Task 1:

Write a program that takes two (02) integer values, does the following operations on those values and displays the result of each operation:

- Addition
- Subtraction
- Multiplication
- Remainder

Task 2:

Write a program that computes the circumference of a circle with radius r = 4, using the formula $C=2\pi r$.

Task 3:

Print the statement **I am in NICE and I am very happy :)**. Print the statement by breaking the statement in two lines.

Task 4:

Calculate the area of rectangle by taking length and width as two inputs from user i.e. length =3 and width = 4.

Task 5:

Write a program to show the following output

```
1  4  7
  2  5  8
    3  6  9
```

Task 6:

Write a program to display the following output:

```
      *
  *       *
*           *
  *       *
      *
```